

Laboratory 5: Bayesian Networks and Autoregressive Language Models

Eshan Saikia
2023A7PS1024G

The accompanying `bn_lab.ipynb` contains the executable probability models, conditional tables, generated text, and validation checks. This report records the required answers and interpretation.

1 From Probability to Language

For a token sequence,

$$P(x_1, \dots, x_T) = P(x_1) \prod_{t=2}^T P(x_t \mid x_1, \dots, x_{t-1}).$$

This decomposition makes generation sequential: after choosing a token, the next token is sampled from a conditional distribution based on the available context. This is the basic autoregressive idea.

2 First-Order Bayesian Network and CPT

The first-order approximation is

$$P(X_t \mid X_1, \dots, X_{t-1}) \approx P(X_t \mid X_{t-1}),$$

corresponding to

$$X_t \perp \{X_1, \dots, X_{t-2}\} \mid X_{t-1}.$$

Thus the graph is

$$X_1 \rightarrow X_2 \rightarrow \dots \rightarrow X_T.$$

The conditional probability is estimated by

$$P(w_j \mid w_i) = \frac{C(w_i, w_j)}{\sum_k C(w_i, w_k)}.$$

For the required contexts:

Context	Nonzero probabilities
the	cat = 0.25, dog = 0.25, mat = 0.1667, rug = 0.1667, park = 0.1667
cat	sat = 0.6667, ran = 0.3333
dog	sat = 0.6667, ran = 0.3333
sat	on = 1.0000
ran	to = 1.0000

All unobserved transitions have probability zero in this unsmoothed model, e.g. $P(\text{on} \mid \text{the}) = 0$.

3 LLM-Assisted Implementation

The first-order prompt was:

Write a simple Python implementation of a first-order autoregressive language model. The model should take tokenised sentences as training data, count transitions between consecutive tokens, construct $P(X_t | X_{t-1})$, display probabilities for a specified previous token, predict the most probable next token, generate a sentence by repeatedly sampling the next token, and stop at <END>. Do not use a machine-learning library or pretrained model. Use ordinary Python data structures and random sampling. Add a safe failure case for a context with no observed continuation.

The generated implementation was inspected before execution. The executable notebook then checked probability normalisation, prediction, generation behaviour, and the second-order extension independently.

4 Validation and Generation

For every observed context,

$$\sum_v P(v | w) = 1.$$

All observed rows passed this check. A total such as 0.87 would indicate an invalid conditional distribution, pointing to a counting, denominator, or normalisation error.

The notebook records 20 first-order sampling generations. Because only one previous token is retained, the model can splice transitions from different training contexts and produce unusual sequences.

For greedy generation, deterministic tie-breaking at **the** selects **cat**, producing

the → **cat** → **sat** → **on** → **the** → ...

The model therefore enters a cycle and does not reach <END>; five bounded greedy traces are recorded in the notebook. Sampling produces more variation because it retains all observed nonzero alternatives.

5 Second-Order Model

The second-order model estimates

$$P(X_t | X_{t-2}, X_{t-1}),$$

with local structure

$$X_{t-2} \rightarrow X_t \leftarrow X_{t-1}.$$

For four tokens,

$$P(X_1, X_2, X_3, X_4) = P(X_1)P(X_2 | X_1)P(X_3 | X_1, X_2)P(X_4 | X_2, X_3).$$

The modification prompt was:

Modify the existing first-order autoregressive model into a second-order model estimating $P(X_t | X_{t-2}, X_{t-1})$. Count observed triples and construct conditional distributions indexed by the previous two tokens. Keep the same training sentences and special tokens. Support next-token prediction and sampling-based generation. Do not replace the model with a neural network or pretrained language model.

The second-order implementation was checked to ensure that observed triples are counted by their two-token context and that generation uses the pair context.

6 First-Order versus Second-Order

Using the 12-token vocabulary and treating each CPT cell as a possible conditional parameter:

Measure	First-order	Second-order
Observed context rows	11	14
CPT cells over vocabulary	132	168
Nonzero context-to-token entries	17	18
Possible context combinations	12	144
Unobserved possible contexts	1	130

The second-order model uses more context and can distinguish situations that share one previous token, but its table is much sparser. On 20 fixed-seed sampling generations, the first-order model produced 11 unique outputs and the second-order model produced 6. The second-order outputs follow the training sentence patterns more closely, while the first-order model more readily combines unrelated transitions.

7 Modern Autoregressive Language Models

Modern autoregressive language models also model

$$P(X_t | X_1, \dots, X_{t-1}).$$

The small models here use explicit CPTs, while modern systems use neural networks to estimate a distribution over a large vocabulary. The shared idea is the conditional next-token objective and chain-rule factorisation; the representation, scale, parameterisation, and learning procedure differ substantially.

8 Questions 1–14

1. **Why is the chain-rule decomposition useful for generation?** It turns generation into repeated conditional prediction: choose one token, then use the resulting context to choose the next.
2. **What independence assumption is made?** Given X_{t-1} , earlier tokens are assumed conditionally irrelevant to X_t .
3. **What is the selected conditional distribution?** The table above gives the nonzero distributions for **the**, **cat**, **dog**, **sat**, and **ran**; all unobserved transitions have probability zero.
4. **Where are transition counts stored?** In a dictionary mapping each context to a ‘Counter’ of observed next tokens.

5. **Where is $P(X_t | X_{t-1})$ computed?** Each context's transition counts are divided by that context's total count during normalisation.
6. **How is the next word chosen?** Greedy mode selects $\arg \max_w P(w | \text{context})$; sampling draws from the complete conditional distribution.
7. **What happens for an unseen context?** The implementation reports that no observed continuation is available rather than inventing a probability.
8. **What would a total of 0.87 mean?** The row is not a valid probability distribution and the implementation must be checked.
9. **Are most-probable predictions always human expectations?** No. The model reflects frequencies and its Markov assumption, so its maximum-probability continuation need not match a person's linguistic expectation.
10. **How do greedy and sampling differ?** Greedy is deterministic and can repeat or cycle; sampling introduces variation by retaining lower-probability alternatives.
11. **How does second-order differ?** The graph has two preceding-token parents for the next token, the CPT is indexed by pairs, more context is available, and more data is required.
12. **Why can more context help and hurt?** More context can make conditional predictions more specific, but the number of possible contexts grows rapidly, making the table sparse with limited data.
13. **Why is a precise LLM specification preferable?** It fixes the intended behaviour and representation before code is generated, making the implementation independently testable.
14. **What did the Bayesian-network view add?** It made token dependencies and independence assumptions explicit, gave a factorisation of the joint distribution, defined conditional probabilities precisely, and provided a direct way to reason about generation and context size.

9 LLM Reflection

The LLM reduced implementation effort by providing a starting structure for counting, normalisation, and generation. Independent checks were still required because syntactically correct code could implement the wrong conditional model or generation rule. The transition counts, probability-normalisation invariant, generation behaviour, and second-order context handling were therefore verified directly in the notebook.

10 Submission Contents

The accompanying notebook contains the first-order and second-order implementations, selected CPTs, all 20 first-order generated sentences, greedy/sampling runs, second-order generations, model-size calculations, and validation checks. No pretrained language model or machine-learning library was used.